

SESSION 13 — Exploratory Data Analysis (EDA)

What is EDA?

EDA means:

Understanding your dataset before analysis or modeling

Why EDA is Important

Before solving any problem, we must know:

- What data we have
 - What patterns exist
 - Are there missing values
 - Are there outliers
-

Types of Analysis

1. Univariate Analysis → one variable
 2. Bivariate Analysis → two variables
 3. Summary Statistics
 4. Correlation Analysis
-

Goal

Convert raw data into insights

```
import pandas as pd
import numpy as np

data = {
    "Region": ["North", "South", "East", "West", "North", "South"],
    "Category": ["A", "B", "A", "B", "A", "B"],
    "Revenue": [100, 200, 150, 300, 250, 400],
    "Profit": [20, 50, 30, 80, 60, 100]
}

df = pd.DataFrame(data)

df
```

	Region	Category	Revenue	Profit
0	North	A	100	20
1	South	B	200	50
2	East	A	150	30
3	West	B	300	80
4	North	A	250	60
5	South	B	400	100

1. Univariate Analysis

Analyzing ONE column at a time

Numerical Column

Use:

```
df["Revenue"].describe()
```

Categorical Column

Use:

```
df["Region"].value_counts()
```

```
# Numerical summary  
df["Revenue"].describe()
```

	Revenue
count	6.000000
mean	233.333333
std	108.012345
min	100.000000
25%	162.500000
50%	225.000000
75%	287.500000
max	400.000000

dtype: float64

```
# Categorical summary  
df["Region"].value_counts()
```

	count
Region	
North	2
South	2
East	1
West	1

dtype: int64

2. Bivariate Analysis

Analyzing relationship between TWO variables

Example

Revenue by Region

Use:

```
groupby()
```

```
df.groupby("Region")["Revenue"].sum()
```

Revenue	
Region	
East	150
North	350
South	600
West	300

dtype: int64

3. Summary Statistics

Get overall dataset statistics

Function

```
df.describe()
```

What it gives:

- Mean
- Std
- Min / Max
- Percentiles

```
df.describe()
```

	Revenue	Profit
count	6.000000	6.000000
mean	233.333333	56.666667
std	108.012345	30.110906
min	100.000000	20.000000
25%	162.500000	35.000000
50%	225.000000	55.000000
75%	287.500000	75.000000
max	400.000000	100.000000

4. Correlation

Measures relationship between numerical variables

Syntax

```
df.corr()
```

Range

- 1 → strong negative
- 0 → no relation
- +1 → strong positive

```
df[['Revenue', 'Profit']].corr()
```

	Revenue	Profit
Revenue	1.000000	0.994153
Profit	0.994153	1.000000

▼ Demo — EDA Insights

Find:

- Total revenue by region
- Average profit
- Most common category

```
print("Revenue by Region:\n", df.groupby("Region")["Revenue"].sum())

print("\nAverage Profit:", df["Profit"].mean())

print("\nMost common category:\n", df["Category"].value_counts())
```

```
Revenue by Region:
Region
East      150
North     350
South     600
West      300
Name: Revenue, dtype: int64

Average Profit: 56.666666666666664

Most common category:
Category
A      3
B      3
Name: count, dtype: int64
```

▼ SESSION 14 — Full Pandas Mini Project

Objective

Apply ALL concepts learned:

- Load data
- Clean data
- Transform data
- Merge datasets
- Pivot data
- Generate insights

Workflow

- 1 Load dataset
- 2 Clean data
- 3 Transform data
- 4 Analyze data
- 5 Generate insights

```
customers = pd.DataFrame({
    "Customer_ID": [1, 2, 3, 4],
    "Region": ["North", "South", "East", "West"]
})

sales = pd.DataFrame({
    "Customer_ID": [1, 1, 2, 3, 4],
    "Product": ["A", "B", "A", "B", "A"],
    "Amount": [100, 200, 300, 150, 250]
```

```
})
```

```
customers
```

	Customer_ID	Region
0	1	North
1	2	South
2	3	East
3	4	West

```
sales
```

	Customer_ID	Product	Amount
0	1	A	100
1	1	B	200
2	2	A	300
3	3	B	150
4	4	A	250

✓ Step 1 — Cleaning

Check for missing values

```
customers.isnull().sum()
```

```
0
Customer_ID  0
Region      0
dtype: int64
```

```
sales.isnull().sum()
```

```
0
Customer_ID  0
Product     0
Amount      0
dtype: int64
```

✓ Step 2 — Merge Data

Combine customer and sales data

```
df = pd.merge(customers, sales, on="Customer_ID")
```

```
df
```

	Customer_ID	Region	Product	Amount
0	1	North	A	100
1	1	North	B	200
2	2	South	A	300
3	3	East	B	150
4	4	West	A	250

Step 3 — Data Transformation

Create new column:

Revenue = Amount

```
df["Revenue"] = df["Amount"]
```

Step 4 — Analysis

Revenue by Region

```
df.groupby("Region")["Revenue"].sum()
```

Revenue	
Region	
East	150
North	300
South	300
West	250

dtype: int64

Step 5 — Pivot Table

Create summary:

Region vs Product

```
df_agg = df.groupby(['Region', 'Product'])['Revenue'].sum().reset_index()
pivot = df_agg.pivot(index="Region", columns="Product", values="Revenue")
pivot
```

Region	Product	
	A	B
East	NaN	150.0
North	100.0	200.0
South	300.0	NaN
West	250.0	NaN

```
pivot = df.pivot_table(index="Region", columns="Product", values="Revenue", aggfunc="sum")
```

pivot

Product	A	B
Region		
East	NaN	150.0
North	100.0	200.0
South	300.0	NaN
West	250.0	NaN

Step 6 — Generate Insights

Answer:

- Which region has highest revenue?
- Which product performs best?

```
print("Revenue by Region:\n", df.groupby("Region")["Revenue"].sum())

print("\nRevenue by Product:\n", df.groupby("Product")["Revenue"].sum())
```

```
Revenue by Region:
Region
East      150
North     300
South     300
West      250
Name: Revenue, dtype: int64

Revenue by Product:
Product
A        650
B        350
Name: Revenue, dtype: int64
```

```
import pandas as pd
import numpy as np

# Customer dataset
customers = pd.DataFrame({
    "Customer_ID": [1,2,3,4,5,6],
    "Name": ["Aman Sharma","Priya Patel","Rahul Verma","Sneha Shah","Karan Mehta","Riya Jain"],
    "Region": ["North","South","East","West","North",None]
})

# Orders dataset
orders = pd.DataFrame({
    "Customer_ID": [1,1,2,3,4,4,5,6,6],
    "Product": ["Laptop","Mouse","Laptop","Tablet","Laptop","Tablet","Mouse","Laptop","Mouse"],
    "Category": ["Electronics","electronics","Electronics","electronics","ELECTRONICS","electronics","electronics","Electronics"],
    "Amount": [50000,2000,55000,30000,60000,25000,1500,70000,np.nan],
    "Order_Date": ["2024-01-01","2024-01-05","2024-02-01","2024-02-10","2024-03-01","2024-03-05","2024-03-10","2024-04-01","2024-04-05"]
})
```

Perform This :

- Data cleaning
- Missing value handling
- String normalization
- Date-time conversion
- Merging datasets
- Feature engineering
- GroupBy analysis
- Pivot reporting

```
customers
```

	Customer_ID	Name	Region
0	1	Aman Sharma	North
1	2	Priya Patel	South
2	3	Rahul Verma	East
3	4	Sneha Shah	West
4	5	Karan Mehta	North
5	6	Riya Jain	None

orders

	Customer_ID	Product	Category	Amount	Order_Date
0	1	Laptop	Electronics	50000.0	2024-01-01
1	1	Mouse	electronics	2000.0	2024-01-05
2	2	Laptop	Electronics	55000.0	2024-02-01
3	3	Tablet	electronics	30000.0	2024-02-10
4	4	Laptop	ELECTRONICS	60000.0	2024-03-01
5	4	Tablet	electronics	25000.0	2024-03-05
6	5	Mouse	electronics	1500.0	2024-03-10
7	6	Laptop	Electronics	70000.0	2024-04-01
8	6	Mouse	electronics	NaN	2024-04-05

Step 1 — Understanding the Data

Before doing anything:

|Always inspect data

Why?

- To understand structure
- To identify missing values
- To check data types

```
customers.info()
orders.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Customer_ID  6 non-null     int64
1   Name         6 non-null     object
2   Region       5 non-null     object
dtypes: int64(1), object(2)
memory usage: 276.0+ bytes
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Customer_ID  9 non-null     int64
1   Product      9 non-null     object
2   Category     9 non-null     object
3   Amount       8 non-null     float64
4   Order_Date   9 non-null     object
dtypes: float64(1), int64(1), object(3)
memory usage: 492.0+ bytes
```

Step 2 — Handling Missing Values

Problem:

- Region has missing value
- Amount has missing value

Why handle?

Missing data leads to wrong analysis

```
# Fill missing Region with "Unknown"
customers["Region"] = customers["Region"].fillna("Unknown")

# Fill missing Amount with mean
orders["Amount"] = orders["Amount"].fillna(orders["Amount"].mean())

customers, orders
```

	Customer_ID	Name	Region
0	1	Aman Sharma	North
1	2	Priya Patel	South
2	3	Rahul Verma	East
3	4	Sneha Shah	West
4	5	Karan Mehta	North
5	6	Riya Jain	Unknown

	Customer_ID	Product	Category	Amount	Order_Date
0	1	Laptop	Electronics	50000.0	2024-01-01
1	1	Mouse	electronics	2000.0	2024-01-05
2	2	Laptop	Electronics	55000.0	2024-02-01
3	3	Tablet	electronics	30000.0	2024-02-10
4	4	Laptop	ELECTRONICS	60000.0	2024-03-01
5	4	Tablet	electronics	25000.0	2024-03-05
6	5	Mouse	electronics	1500.0	2024-03-10
7	6	Laptop	Electronics	70000.0	2024-04-01
8	6	Mouse	electronics	36687.5	2024-04-05

Step 3 — String Cleaning

Problem:

Category column has:

Electronics
electronics
ELECTRONICS

Why fix?

Same category appears different → wrong grouping

```
orders["Category"] = orders["Category"].str.lower()
```

Step 4 — Date-Time Conversion

Problem:

Dates are strings

Why convert?

Required for time analysis

```
orders["Order_Date"] = pd.to_datetime(orders["Order_Date"])

orders["Year"] = orders["Order_Date"].dt.year
orders["Month"] = orders["Order_Date"].dt.month

orders.head()
```

	Customer_ID	Product	Category	Amount	Order_Date	Year	Month
0	1	Laptop	electronics	50000.0	2024-01-01	2024	1
1	1	Mouse	electronics	2000.0	2024-01-05	2024	1
2	2	Laptop	electronics	55000.0	2024-02-01	2024	2
3	3	Tablet	electronics	30000.0	2024-02-10	2024	2
4	4	Laptop	electronics	60000.0	2024-03-01	2024	3

Step 5 — Merge Data

Why?

Customer info and order info are separate

We combine them to analyze fully

```
df = pd.merge(customers, orders, on="Customer_ID")  
  
df.head()
```

	Customer_ID	Name	Region	Product	Category	Amount	Order_Date	Year	Month
0	1	Aman Sharma	North	Laptop	electronics	50000.0	2024-01-01	2024	1
1	1	Aman Sharma	North	Mouse	electronics	2000.0	2024-01-05	2024	1
2	2	Priya Patel	South	Laptop	electronics	55000.0	2024-02-01	2024	2
3	3	Rahul Verma	East	Tablet	electronics	30000.0	2024-02-10	2024	2
4	4	Sneha Shah	West	Laptop	electronics	60000.0	2024-03-01	2024	3

Step 6 — Feature Engineering

Create new column:

Revenue = Amount

Why?

Makes analysis clearer

```
df["Revenue"] = df["Amount"]
```

Step 7 — GroupBy Analysis

Question:

Revenue by Region

Why?

Understand which region performs best

```
df.groupby("Region")["Revenue"].sum()
```

Revenue	
Region	
East	30000.0
North	53500.0
South	55000.0
Unknown	106687.5
West	85000.0

dtype: float64

Step 8 — Multi-Level Grouping

Question:

Revenue by Region and Category

Why?

More detailed insights

```
df.groupby(["Region", "Category"])["Revenue"].sum().reset_index()
```

	Region	Category	Revenue
0	East	electronics	30000.0
1	North	electronics	53500.0
2	South	electronics	55000.0
3	Unknown	electronics	106687.5
4	West	electronics	85000.0

Step 9 — Pivot Table

Goal:

Create report like Excel

Region vs Product

```
pivot = df.pivot_table(
    index="Region",
    columns="Product",
    values="Revenue",
    aggfunc="sum"
)
pivot
```

	Product	Laptop	Mouse	Tablet
Region				
East		NaN	NaN	30000.0
North		50000.0	3500.0	NaN
South		55000.0	NaN	NaN
Unknown		70000.0	36687.5	NaN
West		60000.0	NaN	25000.0

Step 10 — Final Insights

Now answer business questions:

- Which region has highest revenue?
- Which product performs best?

```
print("Revenue by Region:\n", df.groupby("Region")["Revenue"].sum())

print("\nRevenue by Product:\n", df.groupby("Product")["Revenue"].sum())
```

```
Revenue by Region:
Region
East      30000.0
North     53500.0
South     55000.0
Unknown   106687.5
West      85000.0
Name: Revenue, dtype: float64
```

```
Revenue by Product:
Product
Laptop   235000.0
Mouse    40187.5
Tablet    55000.0
Name: Revenue, dtype: float64
```

Final Conclusion

We performed:

- Data cleaning
- Missing value handling
- String normalization
- Date-time conversion
- Merging datasets
- Feature engineering
- GroupBy analysis
- Pivot reporting

Key Learning

Pandas is a complete data analysis tool

Real-world workflow = combination of all steps

===== QUESTIONS: MATPLOTLIB & SEABORN (DATASET BASED) =====

Use the customers and orders dataset (after adding 500 dummy rows)

Q1. Convert 'Order_Date' column to datetime format.

Then extract Month and create a new column 'Order_Month'.

Q2. Using matplotlib, create a line plot showing total sales (Amount) per month.

Q3. Create a bar chart (matplotlib) showing total sales for each Product.

Q4. Create a pie chart (matplotlib) to show proportion of sales by Category.

Q5. Handle missing values in 'Amount' column (fill or drop).

Then re-plot any one graph and observe the difference.

Q6. Using seaborn, create a countplot showing number of orders per Region.

Q7. Create a seaborn barplot showing average Amount spent per Product.

Q8. Create a scatterplot between Order_Month and Amount.

Add hue using Product to differentiate points.

Q9. Create a boxplot using seaborn to show distribution of Amount for each Product.

Q10. Create a violin plot for Amount grouped by Region.

Q11. Convert all values in 'Category' column to lowercase.

Then create a countplot to show category distribution.

Q12. Create a histogram (matplotlib or seaborn) for Amount column.

Q13. Use seaborn's pairplot on numerical columns (if applicable).

Observe relationships between variables.

Q14. Apply different seaborn styles (darkgrid, whitegrid) and compare any one plot.

Q15. Create a combined plot:

- Use matplotlib to plot total monthly sales
- Add markers, title, labels, and legend

BONUS (Optional):

Try to find which Region contributes highest sales and visualize it.

```
=====
```

```
# ===== SOLUTIONS =====

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Merge datasets (needed for Region-based questions)
df = pd.merge(orders, customers, on="Customer_ID", how="left")

# Q1. Convert Order_Date and extract month
df["Order_Date"] = pd.to_datetime(df["Order_Date"])
df["Order_Month"] = df["Order_Date"].dt.month

# Q5. Handle missing values (fill with mean)
df["Amount"] = df["Amount"].fillna(df["Amount"].mean())
```

```

# Q11. Convert Category to lowercase
df["Category"] = df["Category"].str.lower()

# ----- MATPLOTLIB -----

# Q2. Line plot (total sales per month)
monthly_sales = df.groupby("Order_Month")["Amount"].sum()
plt.plot(monthly_sales.index, monthly_sales.values, marker='o')
plt.title("Total Sales per Month")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()

# Q3. Bar chart (sales per product)
product_sales = df.groupby("Product")["Amount"].sum()
plt.bar(product_sales.index, product_sales.values)
plt.title("Sales by Product")
plt.xlabel("Product")
plt.ylabel("Sales")
plt.show()

# Q4. Pie chart (sales by category)
category_sales = df.groupby("Category")["Amount"].sum()
plt.pie(category_sales, labels=category_sales.index, autopct='%1.1f%%')
plt.title("Sales by Category")
plt.show()

# Q12. Histogram
plt.hist(df["Amount"], bins=10)
plt.title("Distribution of Amount")
plt.xlabel("Amount")
plt.ylabel("Frequency")
plt.show()

# Q15. Combined plot
plt.plot(monthly_sales.index, monthly_sales.values, marker='o', label="Sales")
plt.title("Monthly Sales Trend")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.legend()
plt.show()

# ----- SEABORN -----

sns.set_style("darkgrid")

# Q6. Countplot (orders per region)
sns.countplot(data=df, x="Region")
plt.title("Orders per Region")
plt.show()

# Q7. Barplot (avg amount per product)
sns.barplot(data=df, x="Product", y="Amount")
plt.title("Average Amount per Product")
plt.show()

# Q8. Scatterplot
sns.scatterplot(data=df, x="Order_Month", y="Amount", hue="Product")
plt.title("Month vs Amount")
plt.show()

# Q9. Boxplot
sns.boxplot(data=df, x="Product", y="Amount")
plt.title("Amount Distribution by Product")
plt.show()

# Q10. Violin plot
sns.violinplot(data=df, x="Region", y="Amount")
plt.title("Amount Distribution by Region")
plt.show()

# Q11. Countplot (category)
sns.countplot(data=df, x="Category")
plt.title("Category Distribution")
plt.show()

# Q13. Pairplot

```

```
sns.pairplot(df[["Amount", "Order_Month"]])
plt.show()

# Q14. Style change
sns.set_style("whitegrid")
sns.barplot(data=df, x="Product", y="Amount")
plt.title("Whitegrid Style Example")
plt.show()

# BONUS: Region with highest sales
region_sales = df.groupby("Region")["Amount"].sum()
sns.barplot(x=region_sales.index, y=region_sales.values)
plt.title("Sales by Region")
plt.show()

# =====
```